

Lab 2 – Neural Network: Perceptron

Course: CS582 – Machine Learning

Topic: Introduction to Neural Networks and the Perceptron Learning Algorithm

Team members: Nomin /618649/, Tumenjargal /618048/, Temuujin /619957/, Temuulen /618643/

Question 1: Threshold Neuron – Computing Outputs

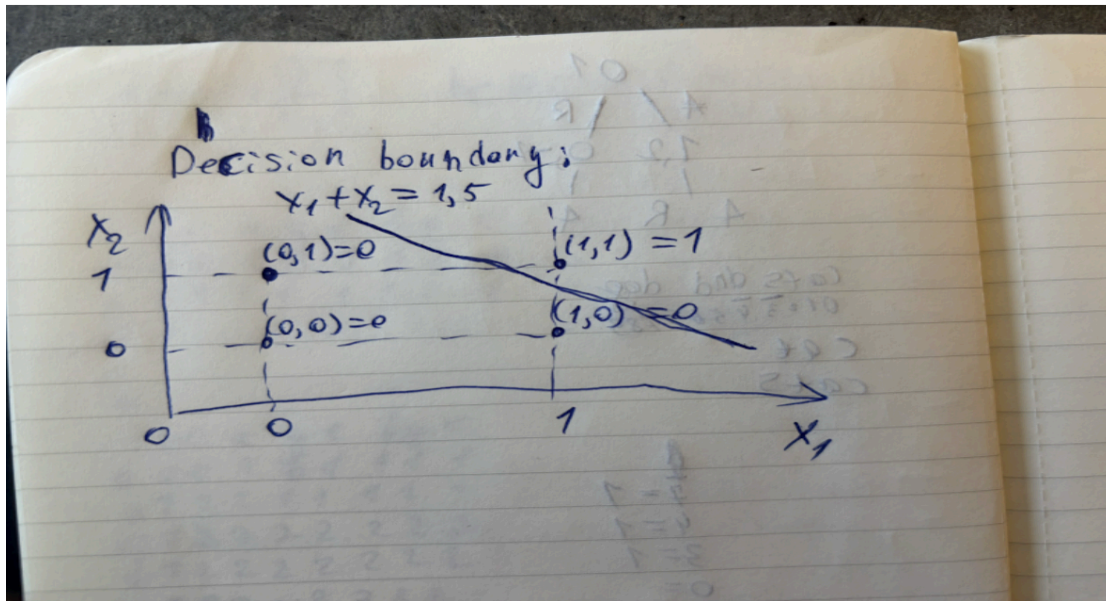
A neuron is given with 2 inputs, weights $w_1 = 1$ $w_2 = 1$ and bias $b = -1.5$.

The rule: add up all the weighted inputs plus the bias. If the result is 0 or more, output 1. Otherwise, output 0. Formula: $z = w_1x_1 + w_2x_2 + b$

x1	x2	$H = w_1 * x_1 + w_2 * x_2 + b$	Output
0	0	$0 + 0 - 1.5 = -1.5$	0
1	0	$1 + 0 - 1.5 = -0.5$	0
0	1	$0 + 1 - 1.5 = -0.5$	0
1	1	$1 + 1 - 1.5 = 0.5$	1

Decision boundary : $x_1 + x_2 = 1.5$

AND Gate - 1 = $(x_1 + x_2 > 1.5)$



Question 2: Perceptrons for NOT, NAND, and NOR

The task is to find weights and bias values that make a perceptron behave like each logic gate. The perceptron outputs 1 when the weighted sum plus bias is 0 or more, and outputs 0 otherwise.

NOT Gate

This gate flips the input. If the input is 0, output 1. If the input is 1, output 0.

Solution: weight = -1 , bias = 0.5

Input	Calculation $h = -x + 0.5$	Result	Output
0	$-1(0) + 0.5 = 0.5$	positive	1
1	$-1(1) + 0.5 = -0.5$	negative	0

All outputs match. The negative weight flips the input, and the bias shifts the threshold.

NAND Gate

This gate outputs 0 only when both inputs are 1. It is the opposite of AND.

Solution: $w_1 = -1$, $w_2 = -1$, bias = 1.5

x1	x2	$h = -x_1 - x_2 + 1.5$	Result	Output
0	0	$0 + 0 + 1.5 = 1.5$	positive	1
1	0	$-1 + 0 + 1.5 = 0.5$	positive	1
0	1	$0 - 1 + 1.5 = 0.5$	positive	1
1	1	$-1 - 1 + 1.5 = -0.5$	negative	0

All outputs match.

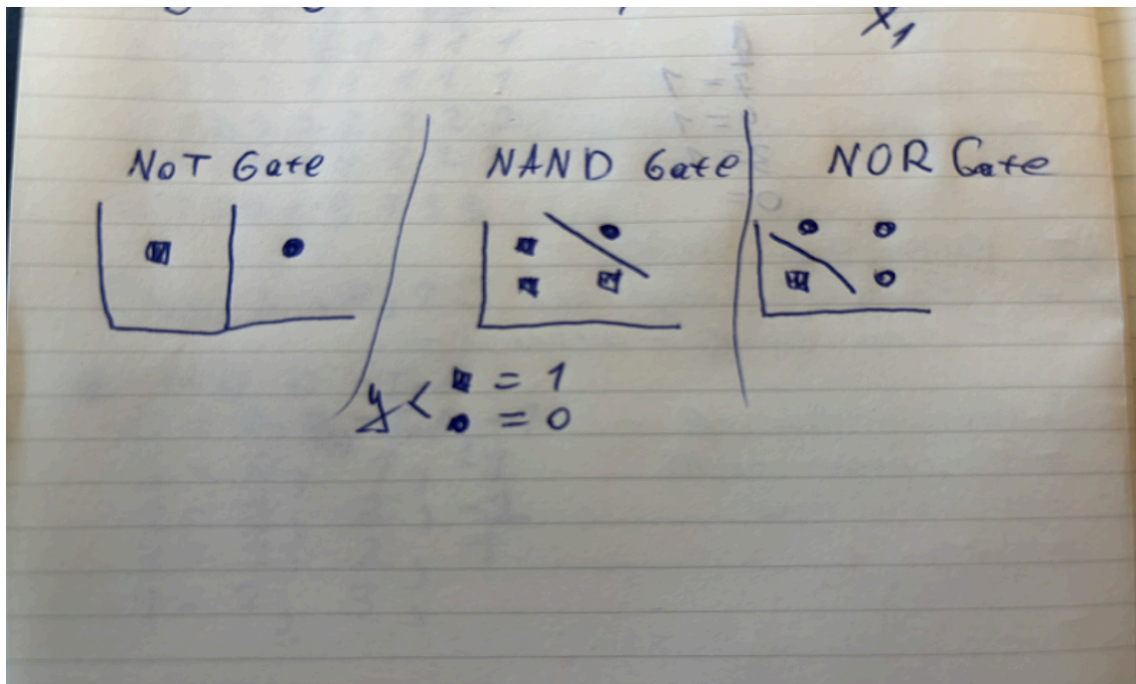
NOR Gate

This gate outputs 1 only when both inputs are 0. It is the opposite of OR.

Solution: $w_1 = -1$, $w_2 = -1$, bias = 0.5

x1	x2	$h = -x_1 - x_2 + 0.5$	Result	Output
0	0	$0 + 0 + 0.5 = 0.5$	positive	1
1	0	$-1 + 0 + 0.5 = -0.5$	negative	0
0	1	$0 - 1 + 0.5 = -0.5$	negative	0
1	1	$-1 - 1 + 0.5 = -1.5$	negative	0

All outputs match.



Question 3: Perceptron Convergence Theorem with

The goal is to prove that if the data can be separated by a straight line, the perceptron will always find it and it will not run forever.

Given: All data points satisfy $\|x_i\| \leq R$ $\|x\| \leq 1$ (instead of $\|x\| \leq 1$). A perfect weight vector w^* exists with margin $\gamma > 0$.

Proof outline in 3 steps:

Step 1 — The weights move toward the right answer. Each time the perceptron makes a mistake and updates, the dot product between the current weights and the perfect weights grows by at least γ . After mistakes:

$$w^{*T} w^{(k)} \geq k\gamma$$

Step 2 — The weights do not grow too fast. Each mistake adds at most R^2 to the squared length of the weight vector (since $\|x_i\|^2 \leq R^2$ and misclassified points contribute a negative or zero middle term). After k mistakes:

$$\|w^{(k)}\|^2 \leq kR^2$$

Step 3 — Combine both bounds. Using the Cauchy-Schwarz inequality ($w^{*T} w^{(k)} \leq \|w^{(k)}\|$):

$$k\gamma \leq \|w^{(k)}\| \leq \sqrt{kR^2}$$

Squaring both sides: $k^2\gamma^2 \leq kR^2$, which simplifies to:

$$k \leq \frac{R^2}{\gamma^2}$$

Conclusion: The perceptron can make at most R^2/γ^2 mistakes before it finds the correct weights. This is a finite number, so the algorithm always stops.

When $R = 1$ (the standard version from Section 3.4.1), this reduces to $k \leq 1/\gamma^2$.

Question 4: Batch vs Sequential Perceptron

The original perceptron code uses **batch update** — it looks at every training example, collects all the errors, and then updates the weights once at the end of each round (epoch).

```

Ch3 > pcn.py
13 class pcn:
16     def __init__(self, inputs, targets):
24         if np.ndim(targets)>1:
25             self.nOut = np.shape(targets)[1]
26         else:
27             self.nOut = 1
28
29         self.nData = np.shape(inputs)[0]
30
31         # Initialise network
32         self.weights = np.random.rand(self.nIn+1, self.nOut)*0.1-0.05
33
34     def pcntrain(self, inputs, targets, eta, nIterations):
35         """ Train the thing """
36         # Add the inputs that match the bias node
37         inputs = np.concatenate((inputs, -np.ones((self.nData, 1))), axis=1)
38         # Training
39         change = range(self.nData)
40
41         for n in range(nIterations):
42
43             self.activations = self.pcnfwd(inputs);
44             self.weights -= eta*np.dot(np.transpose(inputs), self.activations-targets)
45
46         # Randomise order of inputs
47         np.random.shuffle(change)

```

The modified version uses **sequential update**. It updates the weights right away after each individual example, so corrections happen faster.

Both versions were tested on logic gates (AND, OR, NAND, NOR). The results are shown below.

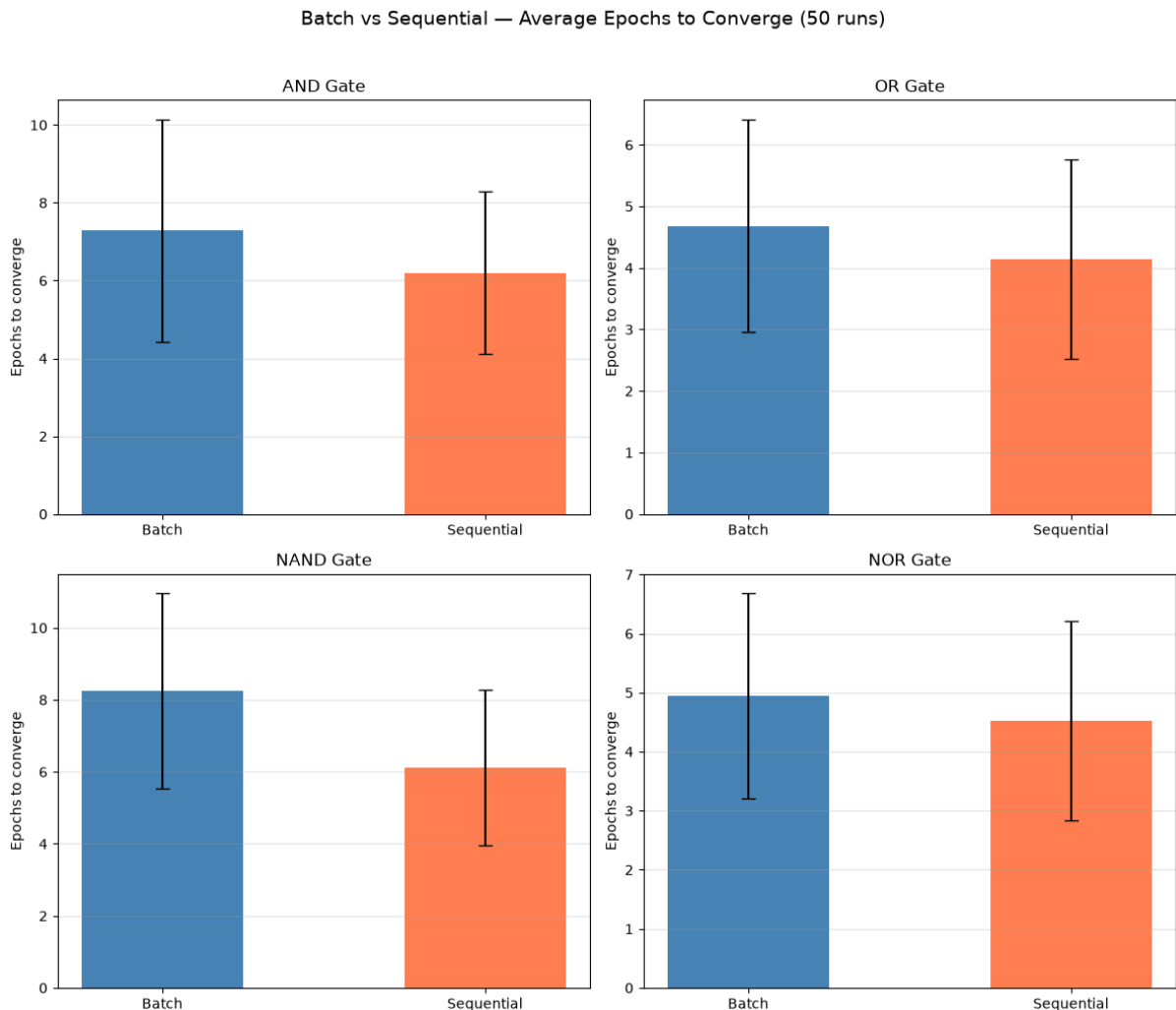
Results on the AND Gate

Method	Epochs to Converge
Batch	7
Sequential	4

Both methods produce the correct output $[0, 0, 0, 1]$ for all four input combinations.

Comparison Across All Logic Gates (averaged over 50 runs)

The bar chart below shows how many epochs each method needs on average:



Observations

- Sequential update learns faster across all four gates because it fixes mistakes immediately instead of waiting until the end of each round.
- Batch update is more stable but slower since it averages all errors before making one change.
- Both methods reach the correct answer for all linearly separable gates. The final weights differ, but both draw a valid decision boundary.

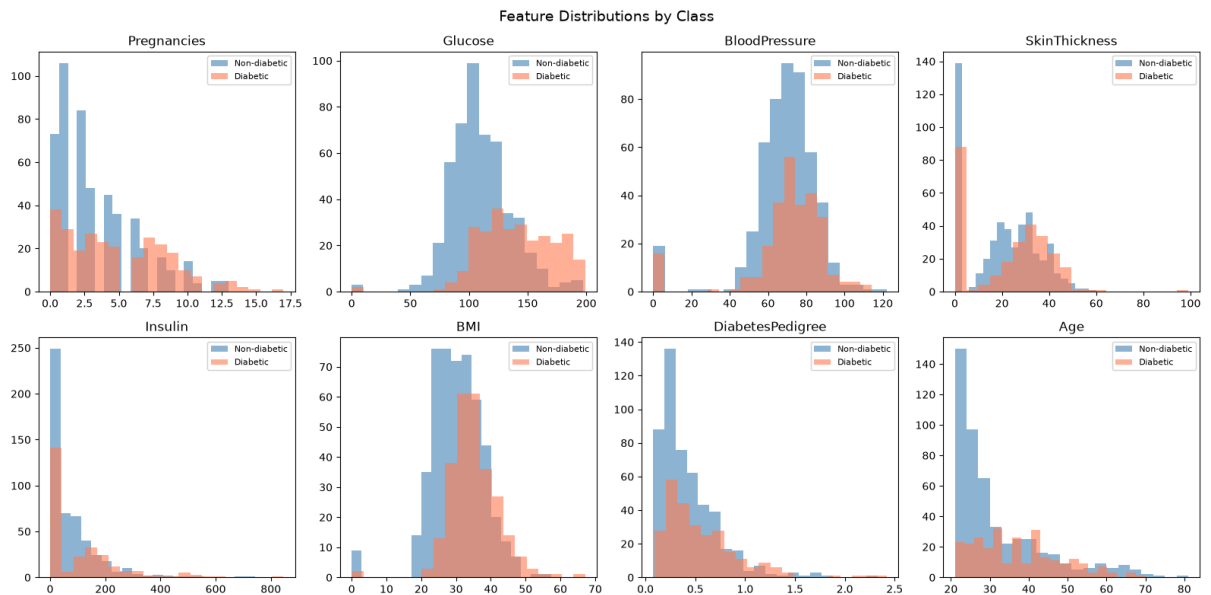
Question 5: Perceptron on the Pima Indians Diabetes Dataset

The Pima Indians Diabetes dataset contains 768 patient records with 8 medical features (glucose level, BMI, age, etc.). The goal is to predict whether a patient has diabetes or not.

Dataset overview:

- 768 total patients: 500 non-diabetic, 268 diabetic
- 8 input features, 1 output (diabetic or not)
- Data was normalized to a 0–1 range so that no single feature dominates the learning
- Split into 614 training samples and 154 test samples

The histograms below show how the features are distributed for each class. Notice that the two groups overlap heavily there is no clean separation between diabetic and non-diabetic patients in any single feature.

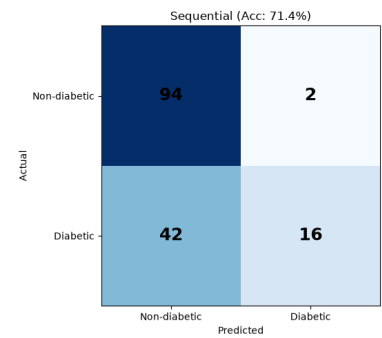
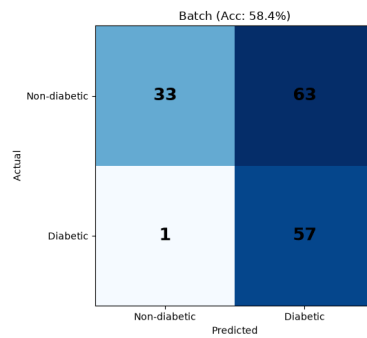
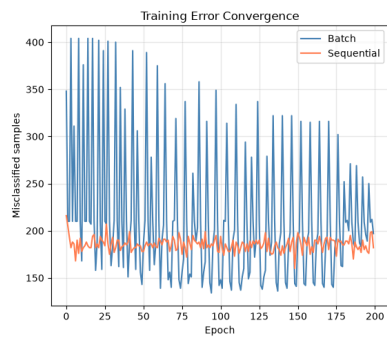


Perceptron Results

Both batch and sequential perceptrons were trained on this dataset for 200 epochs.

Method	Epochs Run	Training Errors (out of 614)	Test Accuracy
Batch	200	197	58.4%
Sequential	200	182	71.4%

The training error convergence and confusion matrices are shown below:



Observations

- The perceptron never reaches zero errors on this dataset. This is because the data is **not linearly separable**; the two classes overlap too much for a single straight line to separate them perfectly.
- The training error keeps bouncing up and down instead of smoothly going down. This is the Perceptron Cycling Theorem when data cannot be separated, the perceptron keeps going in circles.
- Sequential update gives better accuracy (71.4%) than batch (58.4%) on this dataset.
- Glucose and BMI are the features that show the most difference between diabetic and non-diabetic groups, as seen in the histograms above.
- Normalizing the data was necessary because some features have very large ranges (Insulin goes up to 846) while others are very small (DiabetesPedigree maxes at 2.4). Without normalization, the large features would take over the learning.